simu

2026-05-26

Run simulation 2 for Toscano & McMurray (2010).

Let x be the observed cue value for a given cue dimension (e.g. VOT);

Let $i \in \{1, \ldots, K\}$ index the K Gaussian components that together form the learner's internal category representation of a given cue dimension;

Let ϕ_i be the mixing weight, frequency of occurrence of the i-th Gaussian;

Let μ_i be its mean cue value of the Gaussian component along the cue dimension;

Let σ_i be its standard deviation of the Gaussian component along the cue dimension;

Let $G_i(x)$ be the likelihood that the i-th Gaussian generated the observation x given that cue dimension,

Let η_ϕ, η_μ, and η_σ be the learning rates for the trial-by-trial updates of $\Delta\phi_i$, $\Delta\mu_i$, and $\Delta\sigma_i$ to the frequency, mean, and standard deviation of each Gaussian

$$G_i(x) = \phi_i \frac{1}{\sqrt{2\pi\sigma_i^2}} \exp\left(-\frac{(x-\mu_i)^2}{2\sigma_i^2}\right) \tag{4}$$

$$M(x) = \sum_i^K G_i(x) \tag{5}$$

Learning rule:

- A crucial innovation from this model is the use of a winner-take-all update rule such that only one Gaussian updates its value on any given trial. This allows the model to suppress unneeded categories and arrive at the correct solution. Without it, the model does not determine the correct number of categories (see McMurray et al., 2009a).

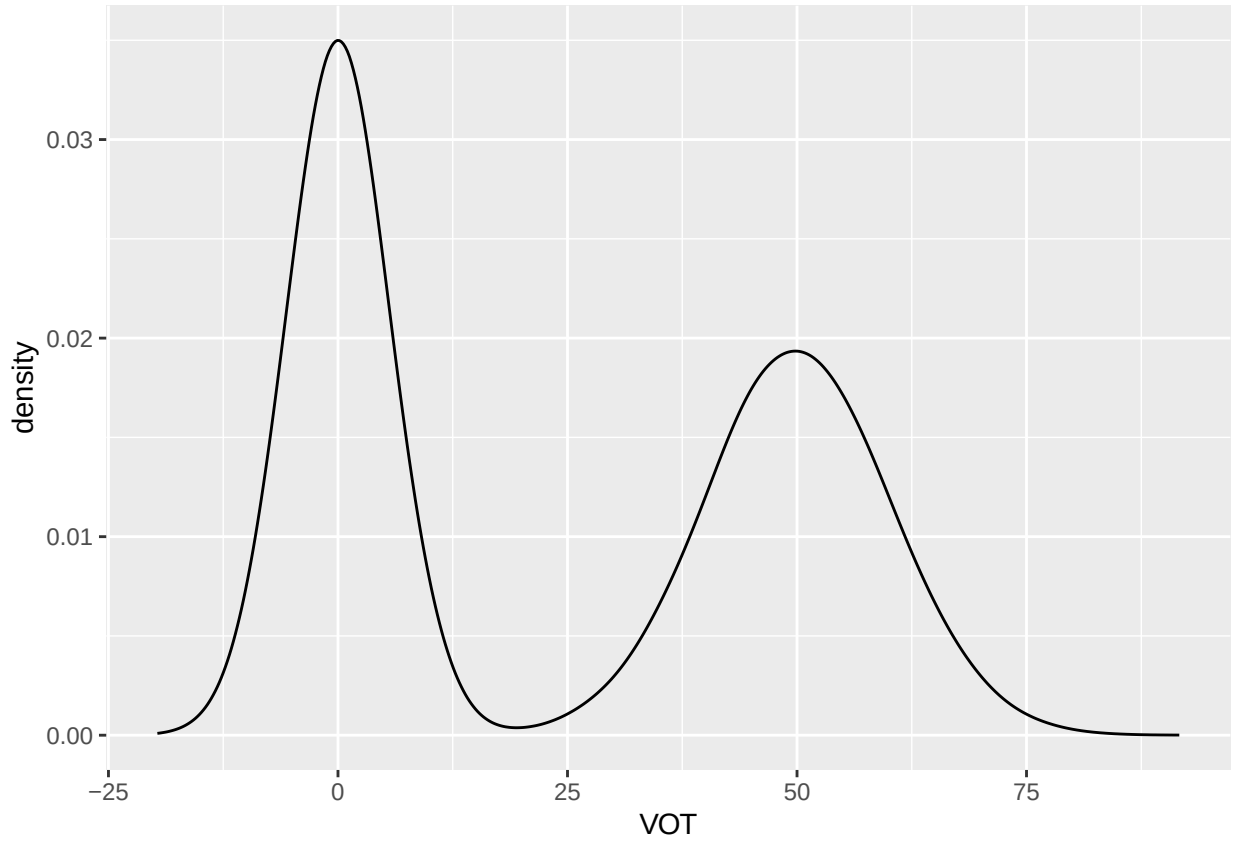
$$\Delta\phi_i = \eta_\phi \frac{G_i(x)}{M(x)} \tag{A2}$$

$$\Delta\mu_i = \eta_\mu \left(\frac{G_i(x)}{M(x)}\right) \frac{(x-\mu_i)}{\sigma_i^2} \tag{A3}$$

$$\Delta\sigma_i = \eta_\sigma \left(\frac{G_i(x)}{M(x)}\right) \left(\sigma_i^{-3}(x-\mu_i)^2 - \sigma_i^{-1}\right) \tag{A4}$$

1 generate data according to table 1

```r
library(ggplot2)

#synthesize data according to table 1
d.vot.voiced <- rnorm(n = 20000, mean = 0, sd = 5)
d.vl.voiced <- rnorm(n = 20000, mean = 188, sd = 45)
d.third.voiced <- rnorm(n = 20000, mean = 260, sd = 10)

d.vot.voiceless <- rnorm(n = 20000, mean = 50, sd = 10)
d.vl.voiceless <- rnorm(n = 20000, mean = 170, sd = 44)
d.third.voiceless <- rnorm(n = 20000, mean = 300, sd = 10)

d.voiced <- data.frame(category = "voiced",VOT= d.vot.voiced,
                       VL = d.vl.voiced, Third  = d.third.voiced)

d.voiceless <- data.frame(category = "voiceless", VOT = d.vot.voiceless,
                          VL = d.vl.voiceless, Third = d.third.voiceless)

d <- rbind(d.voiced, d.voiceless)

# sanity check
ggplot(d, aes(x = VOT)) +
  geom_density()
```



2 build up the model architecture for a single cue (e.g. VOT)

```r
# gap of understanding
mog.object <- function(K = 20, mu.m = 25, mu.sd = 75, sig = 3, seed = 42){
  list(mu = rnorm(K, mu.m, mu.sd),
    sigma = rep(sig, K),
    phi = rep(1 / K, K))
}

# Get G(x) and M(x) according to equation 4 and 5
mog.G <- function(mog.object, x) {
  mog.object$phi * dnorm(x, mean = mog.object$mu, sd = mog.object$sigma)
}
mog.M <- function(mog.object, x) sum(mog.G(mog.object, x))

mog.post <- function(mog.object, x){
  G <- mog.G(mog.object, x)
  M <- mog.M(mog.object, x)
  G/M
}

# learning rule according to A2-A5
# eta is for learning rate
# q.what is the use of sigma.min, sigma.max
mog.update <- function(mog.object, x, eta.mu = 1, eta.sigma = 1, eta.phi = 0.001){
  r <- mog.post(mog.object, x)
  mu.old <- mog.object$mu
  sg.old <- mog.object$sigma

  # A3
  mog.object$mu <- mu.old + eta.mu * r * (x - mu.old) / sg.old^2
  # A4
  sig.new <- sg.old + eta.sigma * r * (sg.old^(-3) * (x - mu.old)^2 - sg.old^(-1))
  mog.object$sigma <- sig.new
  # ai suggests an extra sigma.max, sigma.min
  #if (!is.na(sigma.max)) sg.new <- pmin(sg.new, sigma.max)
  #mog.object$sigma <- pmax(sg.new, sigma.min)

  # A2, winner-take-all, so that unnecessary categories don't get updated
  winner <- which.max(r)
  mog.object$phi[winner] <- mog.object$phi[winner] + eta.phi * r[winner]
  mog.object$phi <- pmax(mog.object$phi, 0)
  mog.object$phi <- mog.object$phi / sum(mog.object$phi)
  mog.object
}

# calculate weight, equation 6, using outer product
mog.weight <- function(mog.object){
  diffs <- outer(mog.object$mu, mog.object$mu, function(a, b) (a - b)^2)
  sigs  <- outer(mog.object$sigma, mog.object$sigma, "*")
  phis  <- outer(mog.object$phi, mog.object$phi, "*")
  0.5*sum(phis * diffs / sigs)
}
```

```r
# sanity check on VOT only updating
mog <- mog.object(K = 20, mu.m = 25, mu.sd = 75, sig = 3)
idx <- sample(nrow(d), 90000, replace = TRUE) # shuffle so voiced/voiceless aren't blocked
for (i in idx)
  mog <- mog.update(mog, d$VOT[i])

mog.weight(mog)
```

```
## [1] 16.0768
```

3 extend one cue to three cues (VOT, VL, combined)

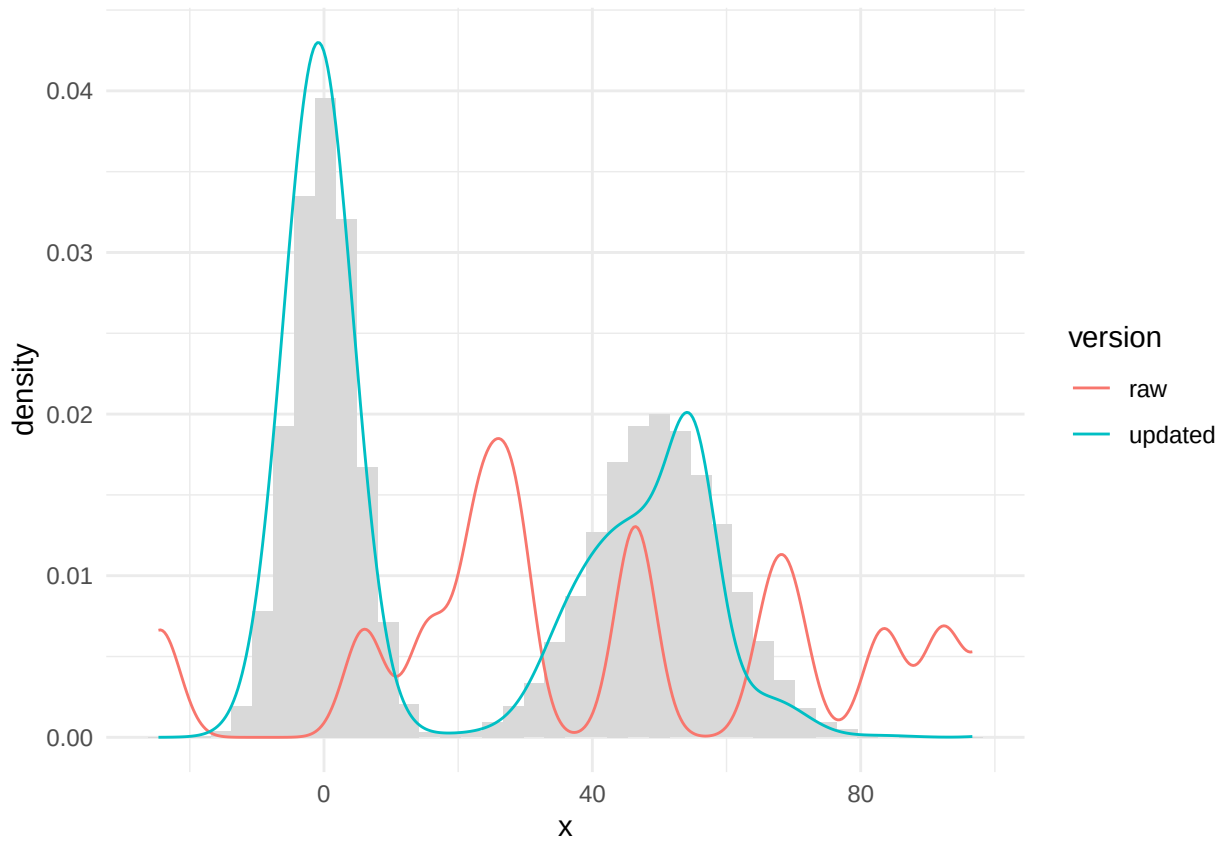
p446. Initial μ values were randomly chosen from a distribution with a mean of 25 and standard deviation of 75 for the VOT dimension and a mean of 179 and standard deviation of 75 for the VL dimension. Initial σ were set to 3 for the VOT dimiension and 10 for the VL dimension. K was set to 20, and initial ϕ were set to 1 / K. Learning rates were set to 1 (η_μ), 1 (η_σ), 0.001 (η_ϕ), and 0.001 (η_p).2 The models were then tested on a range of VOTs (0–40 ms in 5 ms steps) and two VLs (125 and 225 ms).

```r
# set initial parameters as somewhere between voiced and voiceless
vot.mog  <- mog.object(K = 20, mu.m = 25,  mu.sd = 75, sig = 3)
vot.mog.raw  <- mog.object(K = 20, mu.m = 25,  mu.sd = 75, sig = 3)

vl.mog <- mog.object(K = 20, mu.m = 179, mu.sd = 75, sig = 10)
comb.mog <- mog.object(K = 20, mu.m = 0,   mu.sd = 1,  sig = 0.2)

vot.gm <- mean(d$VOT); vot.gs <- sd(d$VOT)
vl.gm  <- mean(d$VL);  vl.gs  <- sd(d$VL)
w.vot <- 0.5
w.vl  <- 0.5

idx <- sample(nrow(d), 5000, replace = TRUE)
for (i in idx) {
  vot.val <- d$VOT[i]
  vl.val  <- d$VL[i]
  # update the two cue MOGs from initial values
  vot.mog <- mog.update(vot.mog, vot.val)
  vl.mog  <- mog.update(vl.mog,  vl.val)
  # normalize the weights so that they sum up to one
  w.vot.raw <- mog.weight(vot.mog)
  w.vl.raw  <- mog.weight(vl.mog)
  tot <- w.vot.raw + w.vl.raw
  w.vot <- w.vot.raw / tot;  w.vl <- w.vl.raw / tot
  # zscore the cue
  z.vot <- (vot.val - vot.gm) / vot.gs
  z.vl  <- (vl.val  - vl.gm ) / vl.gs
  x.comb <- w.vot * z.vot - w.vl * z.vl
  # update the combined ver as well
  comb.mog <- mog.update(comb.mog, x.comb)
}
```

```
## Warning in dnorm(x, mean = mog.object$mu, sd = mog.object$sigma): NaNs produced
```

```
## Warning in dnorm(x, mean = mog.object$mu, sd = mog.object$sigma): NaNs produced
```

```r
xs <- seq(min(d$VOT) - 5, max(d$VOT) + 5, length.out = 400)

# add a VOT axis
plot.df <- rbind(
  data.frame(x = xs, version = "raw", density = sapply(xs, function(x) mog.M(vot.mog.raw, x))),
  data.frame(x = xs, version = "updated", density = sapply(xs, function(x) mog.M(vot.mog,x))))

ggplot() +
  geom_histogram(data = data.frame(x = d$VOT), aes(x = x, y = after_stat(density)), fill = "grey85",bins
  geom_line(data = plot.df, aes(x = x, y = density, colour = version)) +
  theme_minimal()
```



The derivation of A2

We want $\frac{\partial \log M(x)}{\partial \mu_i}$.

Apply the log rule

$$\frac{\partial \log M}{\partial \mu_i} = \frac{1}{M} \cdot \frac{\partial M}{\partial \mu_i} = \frac{1}{M} \cdot \sum_j \frac{\partial G_j}{\partial \mu_i}$$

And as G_j only contains μ_j, not the other μ's. So $\frac{\partial G_j}{\partial \mu_i} = 0$ and only the $j = i$ term survives

$$\frac{\partial G_j}{\partial \mu_i} = \frac{\partial G_i}{\partial \mu_i} = \phi_i \cdot \frac{1}{\sqrt{2\pi\sigma_i^2}} \cdot \frac{\partial}{\partial \mu_i} \left[\exp! \left(-\frac{(x-\mu_i)^2}{2\sigma_i^2} \right) \right]$$

Apply the exponential rule and the chain rule we get the following

$$u(\mu_i) = -\frac{(x-\mu_i)^2}{2\sigma_i^2} \frac{du}{d\mu_i} = -\frac{1}{2\sigma_i^2} \cdot \frac{d}{d\mu_i} \left[(x-\mu_i)^2 \right]$$

apply chain rule once again

$$\frac{d}{d\mu_i}(x-\mu_i)^2 = 2(x-\mu_i) \cdot \frac{d}{d\mu_i}(x-\mu_i) = 2(x-\mu_i) \cdot (-1) = -2(x-\mu_i) \frac{du}{d\mu_i} = -\frac{1}{2\sigma_i^2} \cdot (-2)(x-\mu_i) = \frac{x-\mu_i}{\sigma_i^2}$$

and plug it back

$$\frac{\partial G_i}{\partial \mu_i} = \phi_i \cdot \frac{1}{\sqrt{2\pi\sigma_i^2}} \cdot \exp\left(-\frac{(x-\mu_i)^2}{2\sigma_i^2} \right) \cdot \frac{x-\mu_i}{\sigma_i^2} \frac{\partial G_i}{\partial \mu_i} = G_i(x) \cdot \frac{x-\mu_i}{\sigma_i^2}$$